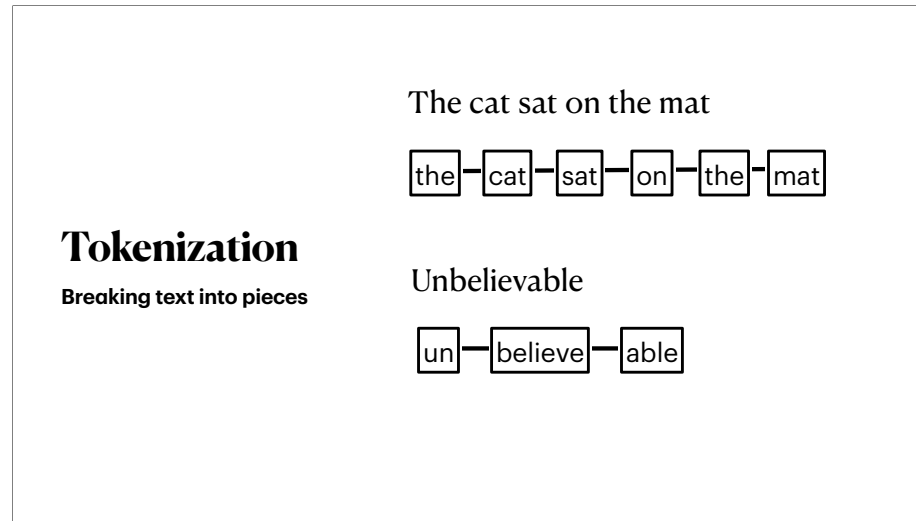


Chapter 3

Tokenization, Embeddings and Attention

Module 1: Foundations of Large Language Models

Every large language model begins with one simple task — turning messy human language into something math can work with. In this chapter we'll unpack how text becomes tokens, how those tokens get turned into meaning, and how attention mechanisms let the model decide what matters most.



Before a model can do anything, it has to break your text into pieces called tokens. Those might be whole words, parts of words, or even single characters. "Unbelievable" might become "un," "believ," and "able." Models don't understand words — they understand numbers. Tokenization is how we get from one to the other.

Turning Tokens into Numbers

Every token gets an ID

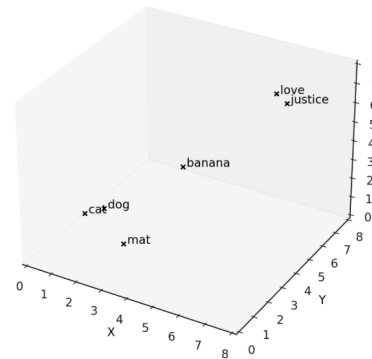
Tokens map to IDs

	Token		ID	
	-----		----	
	The		101	
	cat		2334	
	sat		678	
	on		55	
	the		101	
	mat		892	

Each token gets assigned a numeric ID from the model's vocabulary — think of it like a very large dictionary, sometimes 50,000 entries or more. "The" might be 101, "cat" might be 2334. These IDs don't carry meaning by themselves — they're just the entry point to something more powerful: embeddings.

What are Embeddings?

Meaning in math space

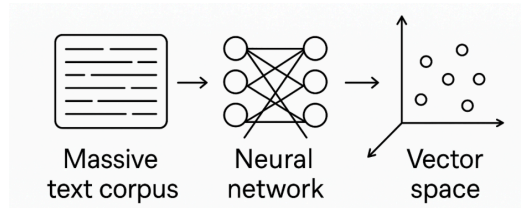


An embedding is a list of numbers — a vector — that represents the meaning of a token. Tokens with similar meanings end up close together in this mathematical space. "Cat" and "dog" cluster near each other. "Justice" and "banana" are far apart. The embedding captures context and relationships that a raw token ID simply can't express.

Another way to think of this is that when you ask a question you want to get the results that are closest to what you meant. If you search your phone for "Starbucks" it shows you the closest results first. This is just like that, on steroids!

How Embeddings are Learned

Patterns in Data

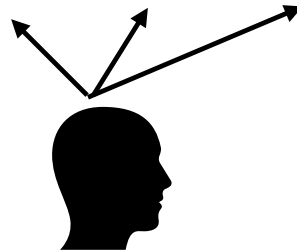


These embeddings aren't hand-coded — the model learns them during pretraining. By reading billions of sentences, it adjusts the embeddings so that tokens appearing in similar contexts drift toward each other. Words used in similar ways end up living near each other in this high-dimensional space. That mathematical closeness is what lets the model generalize and make associations.

The Attention Problem

Deciding where to focus

This is a long **sentence** with some **important** words about **grammar**



Once we have embeddings, we have another problem — not all tokens are equally important. When predicting the next word, the model needs to know which earlier words to focus on. Attention mechanisms do exactly that — they mathematically measure the relationships between tokens and assign more weight to the relevant ones.

Self-Attention in Action

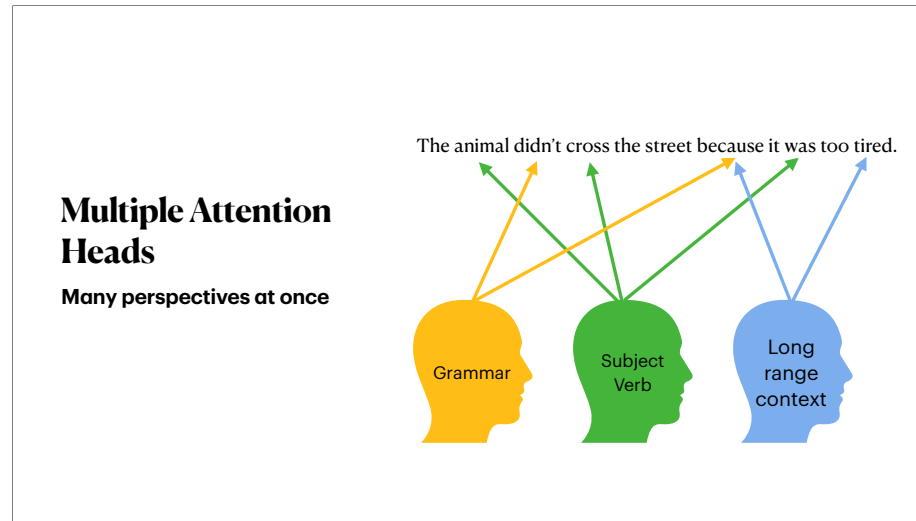
Tokens Looking at Each Other

The animal didn't cross the street

because it was too tired.



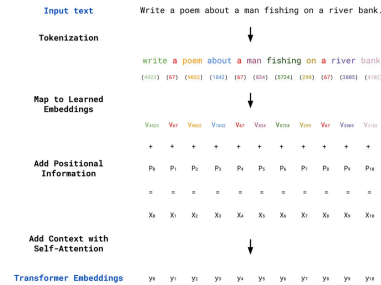
Self-attention means every token looks at every other token to figure out what to emphasize. In the sentence "The animal didn't cross the street because it was too tired" — the model uses attention to connect "it" back to "animal," not "street." That's how it maintains context and coherence across a sentence.



LLMs don't just run attention once — they run it many times in parallel, with each head learning a different type of relationship. One might track grammar, another the topic, another long-range dependencies. Combining all those perspectives is what lets the model capture layered, subtle meaning.

The Output Loop

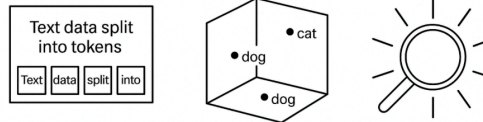
All together now



All three pieces work together on every single generation step. Text comes in, gets tokenized, tokens become embeddings, attention interprets the relationships, and the model predicts the next token. Then it does it again. And again. Until your response is complete. That loop is what turns math into meaning.

Recap and Reflection

From symbols to semantics



Tokenization breaks language into building blocks. Embeddings turn those blocks into meaning. Attention determines what to focus on. Together they let LLMs produce coherent, context-aware responses. Next up we'll look at the limits of this process — and why models still get things wrong.